

SuSchedule-r

A Retrieval-Augmented, Tool-Using Course-Planning Agent for Sabanci University

CS 455 / CS 555 - Large Language Models · Final Project Report · Sabanci University

Track: CS 455 Project

Authors: - Kerem Sirtikizil, 32298 - Cagan Cakir, 32254 - Eray Cagan Ozdemir, 32136

Abstract

SuSchedule-r is a course-advising assistant that grounds a large-language-model agent in Sabanci University's real course catalogue, prerequisite graph, degree requirements, and weekly section offerings. The system combines a FAISS-backed two-stage semantic retriever (BGE bi-encoder plus optional cross-encoder re-ranker over **817 courses**) with a GPT-4o **ReAct** tool-calling agent and an interactive web UI whose schedule builder renders a weekly calendar with live conflict detection.

We evaluate the system on four axes with labeled, reproducible test sets: retrieval quality, time-conflict detection, degree-requirement accuracy, and end-to-end answer grounding. The bi-encoder retriever reaches **Hit@5 = 0.977** and **Recall@10 = 0.966**; the conflict checker is **100% accurate** on labeled meeting pairs and returns **0 unsound schedules** over corequisite-expanded real course bundles; and the ReAct agent grounds **11/12** end-to-end answers, with the single failure isolated to a provisional open-ended recommendation label. We also report negative results honestly: the cross-encoder re-ranker hurts short topical retrieval metrics, the graph requirement engine over-counts remaining required credits by **4 SU** because it lacks a course-substitution table, and future-term schedules use a proxy offering term until the registrar publishes the target term.

1. Introduction

1.1 Problem

Each semester, a Sabanci student needs to choose courses while balancing degree requirements, prerequisites, credit limits, and weekly meeting conflicts. The decision is not just a search problem. A course may be relevant to a student's interests but unavailable, ineligible because of prerequisites, outside the student's degree requirement pool, or impossible to fit into an existing schedule because of a lecture, lab, or recitation conflict.

General-purpose LLMs are fluent but unreliable in this domain. They can invent course codes, confuse course titles, miss recitations, or ignore a student's actual degree audit. For an advising assistant to be useful, factual claims must come from data-backed tools rather than from model memory.

1.2 Design Principle

Our design principle is:

Use the LLM for language, intent, explanation, and tool orchestration; use deterministic code for factual constraints.

The ReAct agent can decide whether to search, inspect a requirement graph, check prerequisites, or validate a schedule. However, the final facts come from deterministic parsers and tools:

- course metadata from scraped Sabanci catalogue pages,
- prerequisite/corequisite expressions parsed into boolean trees,
- per-degree and per-cohort requirement graphs,
- Degree Evaluation parsing for the student's official audit,
- FAISS retrieval over course descriptions,
- timetable conflict detection over normalized meeting intervals.

1.3 Contributions

1. **A grounded course-advising system** for Sabanci University that combines RAG, graph-based requirements, transcript/degree-evaluation context, and timetable reasoning.
2. **A GPT-4o ReAct agent** with a 21-tool toolbox for retrieval, requirements, eligibility, minors, science/engineering credits, timetable construction, and current-schedule checking.
3. **A deterministic data layer:** scrapers, prerequisite parser, degree graph builder, offering parser, requirement evaluator, and schedule conflict engine.
4. **A reproducible evaluation suite** with human-labeled retrieval and agent datasets, local deterministic tests, live agent tests, ablations, and honest error analysis.

2. Data and Knowledge Base

All university knowledge is derived from public Sabanci catalogue/registration pages and the student's own uploaded academic records or Degree Evaluation export.

Asset	Contents	Current scale
Course catalogue	code, title, description, SU credit, ECTS, prerequisites, corequisites, degree sections, seasons	817 courses
Active RAG metadata	one row per indexed course	817 rows
FAISS index	BGE-base course embeddings	817 x 768 vectors
Full prerequisite graph	parsed prerequisite dependencies	721 nodes, 729 edges
Degree/cohort graphs	per-program, per-admit-year requirement slices	14 programs x 5 cohorts x 4 sections
Degree graph files	JSON + gpickle requirement graphs	268 JSON, 268 gpickle
Offerings	section meetings, CRNs, labs, recitations	terms 202401-202503
Student audit	official Banner Degree Evaluation / transcript-derived profile	BSCS-DM sample

2.1 Scraping and Preprocessing

The scraper pipeline is intentionally deterministic and cache-backed. It discovers degree pages, course pools, and course-detail pages; parses course metadata; builds prerequisite graphs; and stores all outputs under `data/`, `courses/`, `degrees/`, `pools/`, and `offerings/`.

The prerequisite parser is a recursive-descent parser. It preserves `AND`, `OR`, parentheses, minimum grades, and concurrent/corequisite conditions as a boolean expression tree. That tree is stored on graph nodes and evaluated later by the eligibility engine. This is important because prerequisites such as:

```
(MATH 204 and CS 300) or instructor consent
```

cannot be represented safely as a flat list of required courses.

Validation results:

- strict prerequisite parse coverage: **426 strict expressions**,
- parser failures: **0**,
- full graph: **acyclic**,
- external prerequisites are preserved as helper nodes when needed.

2.2 Requirement Graphs

The full graph is useful for diagnostics, but normal planning does **not** use the whole graph. The agent selects scoped graphs from:

```
data/degree_graphs/{PROGRAM}/{COHORT}/{SECTION}.gpickle
data/degree_graphs/{PROGRAM}/{COHORT}/{SECTION}.json
```

Examples:

```
data/degree_graphs/BSCS-DM/202201/Required.gpickle
data/degree_graphs/BSCS-DM/202201/Core_Elective.gpickle
data/degree_graphs/BSCS-DM/202201/Area_Elective.gpickle
data/degree_graphs/BSCS-DM/202201/Free_Elective.gpickle
```

This keeps retrieval and requirement reasoning efficient. `in_slice=True` nodes are actual members of the selected requirement/elective pool; upstream prerequisite-only nodes are retained only as validation helpers.

3. System Architecture



3.1 RAG Pipeline

The RAG pipeline has two stages:

1. **Bi-encoder first stage.** BAAI/bge-base-en-v1.5 encodes the query and searches embeddings/su_courses.index, a FAISS IndexFlatIP index over L2-normalized vectors.
2. **Cross-encoder re-ranking.** BAAI/bge-reranker-base can re-rank the first-stage candidates.

FAISS is mandatory. If the FAISS index is missing or inconsistent with the metadata/embedding matrix, the retriever raises an error instead of silently switching to a NumPy search backend. Filtered retrieval uses FAISS IDSelectorBatch, so degree, section, subject, and eligible-course scopes are enforced inside the vector search.

For normal chat questions such as "What does CS 412 cover?" or "Suggest courses about theoretical CS", the model rewrites the latest user message into internal retrieval queries, retrieves catalog context, discards the rewritten queries, and answers the original user message.

3.2 ReAct Agent

The primary runtime mode is a single **tool-calling ReAct agent**, not a multi-agent system. GPT-4o receives a stable system prompt, recent chat history, structured student context, and 21 available tools. It can call tools for factual information and then produce a final answer.

The loop is capped at **8 iterations** per turn. Conversation history is capped by turn count, while structured student state is kept outside history so it survives trimming. The model is proactive through options, but `set_plan()` is only allowed after the user explicitly asks to create or commit a plan.

3.3 Deterministic Guardrails

The LLM is not trusted to validate hard constraints. These are checked by code:

- `eligibility.py` evaluates prerequisite/corequisite boolean trees.
- `requirements.py` computes remaining required/core/area/free courses.
- `degree_eval.py` parses official Degree Evaluation HTML and reads official credit gaps.
- `timetable.py` detects time overlaps and searches feasible section combinations.
- `react_agent.py` repairs unsupported course titles and schedule claims using tool traces.
- `set_plan()` refuses to commit an invalid plan.

3.4 Web UI

The FastAPI + vanilla JavaScript UI has:

- chat interface with trace visibility,
- file upload for transcript JSON/PDF or Degree Evaluation HTML,
- schedule builder with real course sections,
- lab/recitation rows included through corequisite expansion,
- weekly calendar rendering,
- current-schedule conflict checking,
- "Check Schedule" agent review grounded in server-computed conflicts.

Because Fall 2026 (202601) does not have real published offerings yet, scheduling uses the most recent same-season proxy term (202501) and labels this clearly.

4. Evaluation Methodology

The project announcement asks for a working system, appropriate metrics, honest error analysis, reproducibility, and a clear README. We evaluate four components using version-controlled datasets in `eval/`.

4.1 Evaluation Runners

Runner	Measures	Data	OpenAI?
<code>python -m eval.run_retrieval</code>	Hit@k, Recall@k, MRR@10, nDCG@10, re-ranker ablation, FAISS backend evidence	<code>eval/retrieval_queries.json</code>	No
<code>python -m eval.run_conflicts</code>	interval-overlap accuracy, corequisite-expanded timetable soundness and feasibility	hand-labeled pairs + real course bundles	No
<code>python -m eval.run_requirements</code>	official Degree Evaluation credit gaps vs graph engine	<code>DEGREE_EVALUATION.html</code>	No
<code>python -m eval.run_agent</code>	answer grounding, required tool/action success, task success, latency, token cost	<code>eval/agent_questions.json</code>	Yes
<code>python -m eval.run_agent --rescore</code>	re-score saved agent answers after label edits	saved raw answers	No

4.2 Human-Labeled Tests

The retrieval and agent datasets are human-labeled.

`eval/retrieval_queries.json` contains **44 natural-language queries** with gold course codes. The labels are intentionally high-precision rather than exhaustive. For example:

Query	Gold examples	Purpose
"I want to learn machine learning"	CS 412 , CS 415	topical CS retrieval
"courses about neural networks and deep learning"	CS 415 , CS 412	AI/deep-learning retrieval
"how operating systems work internally"	CS 307	course-content retrieval
"designing and querying databases"	CS 306	database retrieval

The dataset uses `review_status`:

- **existing**: 34 reviewed labels used for stable headline analysis,
- **provisional**: 10 exploratory labels that should be reviewed/expanded before being treated as final benchmark labels.

`eval/agent_questions.json` contains **12 conversation questions** with structured checks: required course codes, forbidden course codes, required terms, credit facts, regex patterns, and required semantic actions. It also marks one ambiguous recommendation case as provisional.

This design makes failures inspectable. If the model recommends an unsupported course, the evaluator can say whether the problem was answer grounding, missing tool use, or a weak/ambiguous label.

5. Results

5.1 Retrieval

The current retriever uses FAISS `IndexFlatIP` with **817 vectors**. The retrieval evaluation confirms **88 FAISS searches** and no NumPy search backend.

Metric	Re-ranker ON	Re-ranker OFF	Delta
Hit@1	0.773	0.909	-0.136
Hit@5	0.955	0.977	-0.023
Recall@5	0.875	0.928	-0.053
Recall@10	0.943	0.966	-0.023
MRR@10	0.859	0.938	-0.079
nDCG@10	0.849	0.922	-0.073

Recall@10 by review status:

Label group	n	Recall@10, re-ranker ON
existing	34	0.985
provisional	10	0.800

The bi-encoder alone outperforms the cross-encoder on all six metrics. This is an important negative result: for short topical catalogue queries, exact semantic embedding search is already strong and the cross-encoder sometimes promotes broadly related but less appropriate courses.

5.2 Conflict Detection and Timetable Soundness

Check	Result
Labeled meeting pairs	12
Accuracy / precision / recall	1.000 / 1.000 / 1.000
Real course bundles	6
Unsound schedules returned	0
Wrong feasibility classifications	0
Missing required lab/recitation picks	0
Feasibility accuracy	1.000

The bundle tests start with lecture choices and then auto-add direct catalog corequisites such as `CS 412R`, `CS 308L`, `MATH 101R`. This directly tests a practical scheduling issue: a lecture-only plan can look conflict-free while its required lab/recitation makes it impossible.

5.3 Degree Requirements

The official Banner Degree Evaluation is treated as ground truth for credit completion.

Section	Min SU	Completed SU	Remaining SU
University Courses	41	41	0
Core Electives	31	31	0
Required Courses	29	26	3
Area Electives	9	9	0
Free Electives	15	15	0
General	125	122	3

The graph engine reports:

- required left: CS 395 , ENS 492 , MATH 212 ,
- required credits left: **7 SU**,
- discrepancy against official Required bucket: **+4 SU**.

This is a conservative over-count caused by missing course-substitution/equivalence logic, discussed below.

5.4 End-to-End ReAct Agent

Metric	Value
Questions	12
Answer grounding accuracy	0.917
Action success accuracy	1.000
Task success accuracy	0.917
Existing-label task success	11/11
Provisional-label task success	0/1
Average latency	8.59 s
Average tokens	6,691
Average tool calls	0.5

The single failed item is the provisional optimization/operations-research recommendation. The model suggested CS 306 and justified it through database query optimization, but the provisional gold labels expected IE/OR-flavored courses such as IE 311 , IE 312 , or IE 313 . We count this as a real failure in the reported 12-question score, while also flagging that the accepted answer set for this broad recommendation needs human review.

5.5 Scoreboard

Axis	Headline result
FAISS retrieval	bi-encoder Hit@5 0.977 , Recall@10 0.966
Re-ranker ablation	cross-encoder hurts all six retrieval metrics
Conflict detection	1.000 accuracy, precision, recall
Timetable scheduler	0 unsound schedules over 6 bundles
End-to-end agent	11/12 grounded, 11/11 on reviewed labels
Requirements	+4 SU conservative required-bucket discrepancy
Prereq parser	100% strict parse coverage, 0 failures

6. Error Analysis

6.1 Cross-Encoder Re-Ranker Hurts Short Queries

We expected the cross-encoder to improve precision. It did not. On the 44-query dataset, the re-ranker reduced Hit@1 from **0.909** to **0.773** and MRR@10 from **0.938** to **0.859**.

Example:

Query	Gold	Re-ranked top-5
"I want to learn machine learning"	CS 412 , CS 415	CS 412 , ECON 495 , ECON 494 , ENT 201 , EE 48009
"courses about neural networks and deep learning"	CS 415 , CS 412	PSY 416 , CS 415 , CS 445 , CS 412 , IF 467

The cross-encoder sometimes rewards generic topical overlap and demotes a course that is semantically and institutionally more relevant. The product can still keep the re-ranker behind a flag for longer, more ambiguous questions, but our measured result says the bi-encoder should be treated as the stronger default baseline for short catalogue queries.

6.2 Requirement Engine Needs Course Equivalences

The official audit does not require the student to take MATH 212 , but the graph engine still marks it open. The likely cause is a substitution: the student satisfied that requirement through a MATH 201 + MATH 202 path accepted by Banner. The current graph stores the catalogue requirement literally and does not yet encode equivalence classes or advisor-approved substitutions.

The engine is conservative: it overstates remaining work rather than understating it. Still, exact graduation advising needs an explicit substitution table.

6.3 Future-Term Scheduling Uses Proxy Offerings

The target term 202601 represents Fall 2026-2027, but the real offerings are not published yet. The scheduler therefore uses the most recent same-season term (202501) as a proxy. This is useful for planning but not registration-final. CRNs and section numbers from proxy terms must not be presented as future facts.

6.4 Grounding Is Not Always Mechanically Traceable

Some correct answers use injected context and do not call tools during the visible ReAct loop. This can be acceptable for latency, but it weakens trace auditability. The next version should require citation-producing tool calls for recommendations and factual course claims, especially for broad questions such as "optimization", "operations", "networking", or "security".

6.5 Evaluation Limits

The evaluation is meaningful but not exhaustive. Retrieval labels are small high-precision sets, not complete relevance judgments. Most end-to-end tests use a BSCS-DM sample student. Provisional labels are separated so they do not quietly pretend to be final ground truth.

7. Reproducibility

The repository is runnable from a clean clone with Python 3.10+.

```
python3 -m venv .env
source .env/bin/activate
pip install -r requirements.txt
cp .env.example .env
# edit .env and add OPENAI_API_KEY for live agent tests
```

The FAISS artifacts are versioned for deterministic demos:

```
embeddings/su_courses.parquet
embeddings/id_map.json
embeddings/su_courses_embeddings.npy
embeddings/su_courses.index
```

Run the local tests:

```
python -m eval.run_retrieval
python -m eval.run_conflicts
python -m eval.run_requirements
```

Run the live agent test:

```
python -m eval.run_agent
```

Start the UI:

```
uvicorn api.main:app --reload --port 8000
```

Run the debug CLI for a demo trace:

```
python -m scheduler.cli \
  --transcript data/transcript_cagan.json \
  --term 202601 \
  --mode react \
  --debug-trace
```

8. Use of LLM Assistants and Team Responsibility

The project announcement explicitly allows LLM assistants but requires disclosure. We used them in two distinct roles.

First, LLMs are part of the **system under study**: GPT-4o runs the ReAct advising loop, GPT-4o is also used by the legacy structured planner path, and GPT-4o-mini/GPT-4o can be used for intent/query rewriting depending on configuration.

Second, coding assistants were used as **implementation aids**. The project structure, problem framing, deterministic parser design, graph/requirement architecture, evaluation goals, and iterative test cases were designed by the team. We then used LLM coding assistants to implement and refactor modules such as the ReAct agent loop, helper functions, UI code, evaluation runners, documentation, and trace/debug tooling. The team reviewed outputs, ran tests, corrected failures, and takes responsibility for the final code and report.

We did not use LLMs to fabricate results. Metrics in this report come from version-controlled scripts and JSON outputs under `eval/`.

9. Conclusion

SuSchedule-r satisfies the CS 455 application/system-building goal: it is a working LLM-powered course scheduler-assistant with RAG, ReAct tool use, deterministic parsing, symbolic validation, a web UI, and reproducible evaluation. The system is more complex than a plain chatbot because the LLM is only one layer in a larger grounded architecture. The strongest engineering result is the separation of responsibilities: the model orchestrates and explains, while deterministic tools own catalogue facts, prerequisites, degree requirements, and schedule conflicts.

The project is not perfect. The re-ranker ablation is negative, the requirement engine needs substitution logic, and broad recommendation questions still need stronger citation enforcement. These limitations are visible because the evaluation suite is designed to expose them. That makes the project ready for submission: it works, it is runnable, it is evaluated, and it reports its failures honestly.

Appendix A - Per-Question Agent Results

Mode: ReAct · model: GPT-4o · transcript: `data/transcript_cagan.json` · term: `202601` · total session tokens: **80,292**.

ID	Kind	Question	Grounded	Latency	Tokens	Tool calls	Review
a01	fact-credit	How many SU credits is CS 412?	yes	2.64s	7,932	1	existing
a02	fact-prereq	What are the prerequisites for CS 301?	yes	2.17s	7,483	1	existing
a03	fact-prereq	What is the prerequisite for CS 307?	yes	1.99s	7,637	1	existing
a04	fact-topic	What does CS 445 cover?	yes	8.83s	8,382	1	existing
a05	fact-prereq	Does CS 411 require a math prerequisite?	yes	2.13s	8,014	1	existing
a06	fact-topic	Is CS 412 a machine learning course?	yes	9.49s	4,022	0	existing
a07	fact-topic	What is CS 307 about?	yes	8.73s	4,115	0	existing
a08	recommendation	Recommend one theoretical CS elective.	yes	21.72s	6,364	0	existing
a09	recommendation	Recommend one course about computer networks.	yes	11.67s	6,211	0	existing
a10	fact-corequisite	Does CS 412 require a recitation?	yes	1.29s	4,466	0	existing
a11	timetable	Can CS 412 and CS 302 be scheduled together?	yes	17.76s	9,276	1	existing
a12	recommendation	I enjoy optimization and operations research. Suggest one course.	no	14.64s	6,390	0	provisional

Appendix B - Main Artifacts

- `README.md` : setup, architecture, file map, tests, demo commands.
- `api/` : FastAPI backend and browser UI.
- `scheduler/` : runtime agent, deterministic engines, retriever, timetable, requirements.
- `scraper/` : data ingestion and graph-building pipeline.
- `data/` : catalog, degree graphs, offerings, engineering/science credit table.
- `embeddings/` : deterministic FAISS RAG artifacts.
- `eval/` : datasets, runners, result JSON, agent transcript.
- `report/` : final report, demo script, slides.